Evelyn

1. (a) 9
15
~~5~~
11
~~6~~

(b)

i	a[i] before	total after	a[i] after
0	2	2	2
1	6	8	8
2	5	13	13
3	8	21	21

Output: 6 ␣ 8

(c) Runtime error. The index should below or equal to 4. 5 is out of the range, so it cannot be ran.

(d) Because he only changes the value of v, not the values ~~of~~ v in the array. He should use index numbers ~~to ea~~ and return the values.

2. (a)
```
public static int countEven(int[] data){
int num=0; for (~~data[i]~~ int i=0; i<data.length; i++){
        if(data[i]%2==0){
          ~~return~~ num=num+1;
        }
        else{
        ~~cont~~ continue;
          }
    }
    return num;}
```

(b)
```
public static double average(int[] data){
double average = 0;
```

write this margi

```
double sum=0;
for (int i = 0; i<data.length; i++){
  sum=sum+ data[i];
}
average = sum/data.length;
return average;
for (int i=0; i<data.length; i++){
  if (data[i]> average){
    return data[i]+" ";
```

3. a. [Di, Eve, Cy]
   3
   Di

b. 18

c.
```
public static void removeBelow(ArrayList<Integer>
nums, int limit){
for (int i=0; i<nums.size(); i++){
if (nums.get(i) < limit ){
nums.remove(i);}
}
}
```

d. (i) Because the elements will shift left after removing some of them from the list. Looping backwards can prevent the elements from being skipped.

(ii) It can only store reference objects, not primitive types